

WEEK-1:

Program 1:

```
GNU nano 8.7.1
#include <stdio.h>
int main(){
printf("Welcome to operating system\n");
return 0;
}
```

```
userhasini@DESKTOP-QDSCC4:~/oslab$ nano hello.c
userhasini@DESKTOP-QDSCC4:~/oslab$ gcc hello.c -o hello
userhasini@DESKTOP-QDSCC4:~/oslab$ ./hello
Welcome to operating system
```

Program 2:

```
GNU nano 8.7.1
#include <stdio.h>
int main(){
printf("Enter Your name:");
scanf("%s",name);
printf("Welcome &s\n",name);
return 0;
}
```

```
userhasini@DESKTOP-QDSCC4:~/oslab$ nano readusername.c
userhasini@DESKTOP-QDSCC4:~/oslab$ gcc readusername.c -o readusername
userhasini@DESKTOP-QDSCC4:~/oslab$ ./readusername
Enter your name: hasini
Welcome hasini
userhasini@DESKTOP-QDSCC4:~/oslab$
```

Program 3:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main(){
fork();
printf("Hello \n");
return 0;
}
```

```
userhasini@DESKTOP-QDSCC4:~$ nano display.c
userhasini@DESKTOP-QDSCC4:~$ gcc display.c -o display
userhasini@DESKTOP-QDSCC4:~$ ./display
Hello
Hello
```

Program 4:

```
#include <stdio.h>
#include <unistd.h>
int main(){
pid_t pid;
pid = fork();
if (pid==0){
printf(" I am Child Process\n");
}
else {
printf("I am parent process\n");
}
return 0;
}
```

```
parent.c:8:1: note: each undeclared identifier is reported only once for each function it appears in
userhasini@DESKTOP-QDSCC4:~$ nano parent.c
userhasini@DESKTOP-QDSCC4:~$ gcc parent.c -o parent
userhasini@DESKTOP-QDSCC4:~$ ./parent
I am parent process
I am Child Process
```

Program 5:

```

GNU nano 8.7.1
#include<stdio.h>
#include<unistd.h>
int main(){
    pid_t pid;
    pid = fork();
    if (pid==0){
        printf("Child PID=%d\n",getpid());
        printf("Parent PID=%d\n",getpid());
    }
    else {
        printf("Parent PID = %d\n",getpid());
    }
    return 0;
}

```

```

userhasini@DESKTOP-QDSCC4:~$ gcc parent.c -o parent
userhasini@DESKTOP-QDSCC4:~$ ./parent
Parent PID = 795
Child PID=796
Parent PID=796

```

```

GNU nano 8.7.1
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main(){
    pid_t pid;
    pid=fork();
    if (pid==0){
        printf("child running");
    }
    else {
        wait(NULL);
        printf("Parent Running After Child\n");
    }
    return 0;
}

```

```

userhasini@DESKTOP-QDSCC4:~$ gcc wait.c -o wait
userhasini@DESKTOP-QDSCC4:~$ ./wait
child runningParent Running After Child

```

Program 7:

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(){
    char cmd[100];
    printf("Enter linux command:");
    scanf("%s", cmd);
    pid_t pid = fork();
    if (pid==0){
        printf("child pid: %d\n",getpid());
        execlp(cmd, cmd, NULL);
        perror("Exec Failed");
    }
    else if(pid > 0){
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Child process completed.\n");
    }
    else
    {
        printf("Fork Failed\n");
    }
    return 0;
}

```

```

userhasini@DESKTOP-QDSCC4:~$ nano child.c
userhasini@DESKTOP-QDSCC4:~$ gcc child.c -o child
userhasini@DESKTOP-QDSCC4:~$ ./child
Enter linux command:ls
Parent PID: 649
child pid: 650
child display file1.txt hello.c parent practical1.c re
child.c display.c hello oslab parent.c practical1.c.save re
Child process completed.
userhasini@DESKTOP-QDSCC4:~$

```

Program 8:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main(){
    int src, dest;
    char buffer[1024];
    int bytes;
    src = open("input.txt", O_RDONLY);
    if(src < 0){
        printf("Cannot open source file\n");
        return 1;
    }
    dest = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    while((bytes = read(src, buffer, sizeof(buffer))) > 0)
    {
        write(dest, buffer, bytes);
    }
    close(src);
    close(dest);
    printf("File copied successfully.\n");
    return 0;
}
```